

# 机器学习笔记

Leonhard Hsiao

2025 年 12 月 19 日

目录

|       |             |   |
|-------|-------------|---|
| 1     | 特征选择和特征降维   | 1 |
| 1.1   | 特征选择        | 1 |
| 1.1.1 | 特征子集生成的搜索策略 | 2 |
| 1.1.2 | 特征评价准则      | 3 |
| 1.1.3 | 特征选择方法      | 6 |

# 1 特征选择和特征降维

在机器学习任务中，原始数据往往拥有极高的维度，进行降维处理主要基于以下四个原因：

- **维度灾难**：随着维度  $d$  的增加，样本空间体积呈指数级膨胀，导致数据在高维空间变得极其稀疏，模型就很难学到规律，而且有可能把噪声当作数据学习。
- **冗余与噪声**：噪声是指某些特征对于预测任务是无用的，还有可能干扰学习过程；某些特征之间存在高度相似性，比如年龄和出生日期，同时考虑他们两个是没有必要的。
- **计算效率**：减少特征数量可以显著降低模型训练和推理时的计算复杂度，节省内存与存储空间。

虽然两者都旨在降低维度，但在处理手段和结果上有本质区别。特征选择是从原始特征集合中选取一个子集，保持特征的原有意义；特征提取类似于线性映射  $\mathbb{R}^n \rightarrow \mathbb{R}^m$ ，把数据从高维的特征空间映射到低维空间，过程会创造新的特征。

## 1.1 特征选择

**定义 1.1** (特征选择问题). 特征选择 (Feature Selection) 是指从原始的  $D$  维特征集合中，挑选出一个包含  $d$  个特征 ( $d < D$ ) 的子集来表达模式的过程。其核心目标是在尽量减少信息损失的前提下：

- 提高数据的压缩率；
- 提升学习机的预测性能（泛化能力）；
- 加快学习速度并降低计算开销。

特征选择通常是一个迭代循环的过程，包含四个核心环节：

1. **子集生成**：按照某种搜索策略产生候选的特征子集。
2. **子集评价**：利用评价准则  $J$  对候选子集进行打分，衡量其优劣。
3. **停止准则**：判断当前搜索是否达到预期目标（如达到指定特征数或评价指标不再提升）。
4. **结果验证**：在独立的测试集上验证最终选定子集的有效性。

### 1.1.1 特征子集生成的搜索策略

从  $D$  维特征中选择  $d$  维特征，其可能的子集个数为组合数：

$$C_D^d = \frac{D!}{(D-d)!d!}$$

当原始维度  $D$  较大时，搜索空间呈指数级增长。例如，当  $D = 100, d = 10$  时，组合数约为  $1.73 \times 10^{13}$ 。为了在有限时间内找到理想子集，需要采用不同的搜索策略。

**1. 穷举搜索 (Exhaustive Search)** 计算特征的所有可能组合并逐一评价。虽然能找到最优子集，但总搜索数  $\sum_{d=1}^D C_D^d = 2^D - 1$  计算量巨大，仅适用于  $D$  非常小的情况。

**2. 单独最优特征组合** 对每个特征  $x_i$  分别进行单独评估，计算其评价得分  $J(x_i)$ ，然后按得分从大到小排序，选取前  $d$  个特征。这样算法极简，计算量极低。但是完全忽略了特征之间的相关性，无法处理特征冗余。在这个算法中，年龄和出生日期这两个特征都会打高分并被选择，但是这就带来了冗余信息。

**3. 序贯搜索算法 (Sequential Search)** 本质上这是一种贪心算法，通过每一步的局部最优选择来逼近全局最优。

---

#### Algorithm 1 前向序贯选择算法 (SFS)

---

**Require:** 特征全集  $X = \{x_1, \dots, x_D\}$ , 目标维度  $d$

**Ensure:** 最优特征子集  $X_{selected}$

- 1: 初始化  $X_{curr} \leftarrow \emptyset$
  - 2: **for**  $k = 1$  **to**  $d$  **do**
  - 3:     寻找  $x_j \in X \setminus X_{curr}$ , 使得  $J(X_{curr} + \{x_j\})$  最大
  - 4:      $X_{curr} \leftarrow X_{curr} \cup \{x_j\}$
  - 5: **end for**
  - 6: **特点:** 一旦特征被加入，后续步骤无法将其删除。
  - 7: **return**  $X_{curr}$
- 

#### 4. 其它序贯变体

- **GSFS (广义 SFS):** 每次不是加入 1 个特征，而是加入  $k$  个特征，使得组合后性能最优。其计算量比标准 SFS 大。

- **SBS (后向序贯选择)**: 从全集开始, 每次剔除一个特征  $x_j \in X_{curr}$ , 使得剩余特征集的评价准则  $J(X_{curr} - \{x_j\})$  最大。其特点是“一旦删除, 无法增加”。
- **GSBS (广义 SBS)**: 每次从当前集合中剔除  $k$  个特征。

**5. L-R 法 (Plus-L Minus-R)** 为了克服 SFS 和 SBS “只进不出”或“只出不进”的单向性缺陷, L-R 法采用交替增减的策略。每次先通过 SFS 增加  $L$  个特征, 再通过 SBS 减少  $R$  个特征。若  $L > R$ , 最终特征数增加; 若  $L < R$ , 最终特征数减少。每次寻优后的集合为  $X_d = X_{d1} + \{x_j\}_L - \{x_i\}_R$ 。广义 L-R ( $ZL, ZR$ ) 更进一步, 增加  $L$  个和减少  $R$  个的过程分  $Z$  步进行, 通过复杂的步长控制来更深层地搜索特征组合空间。

### 1.1.2 特征评价准则

**定义 1.2** (特征评价准则). 特征评价准则是一个函数  $J(X_{selected})$ , 用于定量评估所选特征子集的分类贡献或表达能力。评价准则主要分为两类:

- **可分性度量**: 利用类别间的几何距离、分布重叠度或统计相关性进行打分, 常用于 Filter 框架。
- **学习算法精度度量**: 直接利用分类器的准确率或错误率作为得分, 常用于 Wrapper 框架。

**基于距离的可分性判据** 这类判据的核心思想是: 好的特征子集应该使得“类内距离尽可能小, 类间距离尽可能大”。这通常依赖于对样本散度矩阵的计算。

**1. 基础数据统计量** 假设特征选择后的样本向量为  $x'_j$ , 第  $i$  类的平均向量为  $m_i$ , 总平均向量为  $m$ , 样本总数为  $n$ 。

- **类内散度矩阵 (Within-class Scatter Matrix)**: 表示第  $i$  类样本内部的离散程度。

$$S_i = \frac{1}{n_i} \sum_{j=1}^{n_i} (x'_j - m_i)(x'_j - m_i)^T$$

- **总类内离散度矩阵 ( $S_w$ )**: 各类别散度矩阵的加权和。

$$S_w = \sum_{i=1}^c P_i S_i \quad (P_i \text{ 为第 } i \text{ 类的先验概率})$$

- **类间散度矩阵 ( $S_b$ )**: 表示各类别中心相对于总平均值的离散程度。

$$S_b = \sum_{i=1}^c (m_i - m)(m_i - m)^T$$

## 2. 常用可分性判据 Fisher 鉴别 $J_f$

- **一维特征 (两类)**: 采用比值形式。

$$J_f = \frac{(m_1 - m_2)^2}{S_1^2 + S_2^2}$$

- **多维特征子集**: 原本也应该是  $J_f = \frac{S_b}{S_w}$ , 但是结果是一个矩阵而不是值, 下面是常用的取值:
  - $J_1 = \text{tr}(S_w + S_b)$ : 总散度。
  - $J_2 = \text{tr}(S_w^{-1} S_b)$ : 类间与类内散度的比率 (最常用)。
  - $J_3 = \ln \frac{|S_b|}{|S_w|}$ : 行列式比值的对数。
  - $J_4 = \frac{\text{tr}(S_b)}{\text{tr}(S_w)}$ : 迹的比值。
  - $J_5 = \frac{|S_b - S_w|}{|S_w|}$ : 差值的比例。

**基于概率分布的可分性判据** 这个判据依赖于类别在特征空间的概率密度函数。比如水果中西瓜这个特定的类别, 水果有很多特征, 重量、含糖量、外表颜色等, 现在只关注重量这个特征, 西瓜可以有不同的重量, 这就构成了一个概率分布, 橘子也有重量, 也有对应的概率分布, 西瓜一般来说重量自然是比橘子大的多, 所以二者概率密度函数重叠并不高。但是对于梨子和苹果, 二者的重量概率分布函数重叠很高, 我们就很难用重量这个特征区分二者。在实际操作中, 我们很难得到真正的概率分布, 所以一般会通过数据假设概率分布函数形式并通过参数估计决定概率分布函数的具体形式。那么现在一个重要的问题就是如何选择一个合理的概率距离准则  $J$ 。一个合理的概率距离准则  $J$  应满足:

1.  $J \geq 0$ 。
2. 当类别完全不重叠 ( $p(x|\omega_1)p(x|\omega_2) = 0$ ) 时,  $J$  达到最大值  $J_{max}$ 。
3. 当类别完全重合 ( $p(x|\omega_1) = p(x|\omega_2)$ ) 时,  $J = 0$ 。

常见的概率距离准则有以下几类:

- **Bhattacharyya 距离**: 衡量概率分布的相似度。

$$J_B = -\ln \int [p(x|\omega_1)p(x|\omega_2)]^{1/2} dx$$

- **Chernoff 距离**: Bhattacharyya 的广义形式。

$$J_C = -\ln \int p^s(x|\omega_1)p^{1-s}(x|\omega_2) dx \quad (0 \leq s \leq 1)$$

- **散度 (Divergence)**: 衡量两个分布之间的不对称性差异。

$$J_D = \int [p(x|\omega_1) - p(x|\omega_2)] \ln \frac{p(x|\omega_1)}{p(x|\omega_2)} dx$$

- **Matusita 距离**:

$$J_M = \left[ \int (\sqrt{p(x|\omega_1)} - \sqrt{p(x|\omega_2)})^2 dx \right]^{1/2}$$

**基于熵的判据 (信息论视角)** 从信息论的角度看, 特征选择的任务可以被视为一个“消灭不确定性”的过程。评价一个特征好坏可以利用“得到这个特征之后, 我对判断的困惑程度增加了多少”来理解。这就是**互信息 (Mutual Information,  $I(X; Y)$ )**。互信息的核心思想是衡量特征  $X$  与类别  $Y$  的**关联程度**, 他的一个核心表达式是:

$$h(x, y) = \frac{p(x, y)}{p(x)p(y)}$$

- **独立情况 (无价值)**: 若  $X$  与  $Y$  完全无关, 则联合概率等于概率乘积 ( $p(x, y) = p(x)p(y)$ ), 比值为 1,  $\log(1) = 0$ , 互信息为 0。
- **相关情况 (有价值)**: 若  $X$  与  $Y$  存在关联, 比值将偏离 1。可以假设有与判断目标  $Y$  完全相关的特征  $X$ , 则离散情况下, 分布式为对角矩阵。这个时候  $p(x, y) = p(x) = p(y)$ , 对角元素为  $1/p(x)$ , 说明特征  $x$  可以很好的决定  $y = x$  时的信息; 对于非对角元素, 结果是 0, 说明完全不提供  $y \neq x$  处的信息。

对于特征  $X$  和类别  $Y$ , 互信息的计算取决于数据的类型:

**(1) 离散情况 (Discrete Case)** 当特征和类别均为离散取值时, 求和是对特征  $X$  的所有可能状态 (或区间) 以及所有类别  $Y$  的组合进行遍历:

$$I(X; Y) = \sum_{x \in \mathcal{X}} \sum_{y \in \mathcal{Y}} p(x, y) \log \frac{p(x, y)}{p(x)p(y)}$$

注: 在实际操作中, 概率  $p(x, y)$  通常通过统计样本落在特定组合格子的频率来估算。

**(2) 连续情况 (Continuous Case)** 若特征  $X$  是连续数值 (如温度、电压), 公式转化为在特征空间上的二重积分:

$$I(X; Y) = \int_{\mathcal{X}} \int_{\mathcal{Y}} p(x, y) \log \frac{p(x, y)}{p(x)p(y)} dx dy$$

注: 由于连续概率密度函数  $p(x, y)$  较难直接获得, 工程上常通过“分箱 (Binning)”将连续特征离散化, 再利用离散公式进行计算。

另外一个重要的判据是条件互信息 (Conditional Mutual Information) 在已知特征  $Z$  的前提下, 特征  $X$  为判断类别  $Y$  提供的额外信息量:

$$I(X; Y | Z) = \sum_{z \in \mathcal{Z}} p(z) \sum_{x \in \mathcal{X}} \sum_{y \in \mathcal{Y}} p(x, y | z) \log \frac{p(x, y | z)}{p(x | z) p(y | z)}$$

这常用于检测特征冗余: 如果已知  $Z$  后,  $X$  提供的额外信息几乎为 0, 则说明  $X$  相对于  $Z$  是冗余特征。

### 1.1.3 特征选择方法

**Filter 方法 (过滤式)** Filter 方法不依赖于后续的学习算法 (如分类器)。它直接根据数据的统计特性 (如距离、相关性、互信息等) 构建评估函数, 对特征子集进行评价。其逻辑是“先过滤, 后训练”, 即先从原始特征中选出最优子集, 再将其输入学习器。其计算复杂度低, 执行效率高, 适用于大规模数据集。但是选择过程与学习算法解耦, 可能导致选出的特征子集在特定模型上性能不是最优, 且容易忽略特征间的冗余信息。

例 1.3 (基于距离评价的特征选择流程). 假设有两类 3 维数据 ( $W_1$  红色,  $W_2$  绿色), 每个类别包含 3 个样本。我们的目标是从 3 个属性  $x_1, x_2, x_3$  中选择最有用的特征。

1. **1 维特征评估**: 对每个特征独立计算 Fisher 判据  $J_f = \frac{(m_1 - m_2)^2}{S_1^2 + S_2^2}$ 。通过观察数据分布 (Slide 40), 发现  $x_2$  在两类间的均值差异最大且类内分布紧凑。计算结果排序为:  $J(x_2) > J(x_3) > J(x_1)$ 。
2. **子集组合评估**: 当需要选择 2 维特征子集时, 计算所有可能组合 (如  $\{x_1, x_2\}, \{x_2, x_3\}$  等) 的矩阵迹判据  $J_2 = \text{tr}(S_w^{-1} S_b)$ 。
3. **结论**: 选择  $J_2$  得分最高的子集 (如示例中的 53.2917 分组合), 该子集在空间中能产生最佳的类别分离度。

例 1.4 (Relief 算法). Relief 算法通过计算特征对于“同类近邻”的凝聚能力和“异类近邻”的区分能力来评估特征权重。



---

**Algorithm 2** Relief 算法流程

---

**Require:** 样本集  $D$ , 迭代次数  $m$ , 阈值  $t$ **Ensure:** 特征权重向量  $w$ 

```
1: 初始化所有特征权重  $w_i \leftarrow 0$ 
2: for  $i = 1$  to  $m$  do
3:   随机抽取一个样本  $x$ 
4:   在  $x$  的同类中寻找最近邻样本  $z^+$  (Near-hit)
5:   在  $x$  的异类中寻找最近邻样本  $z^-$  (Near-miss)
6:   for 每个特征  $j$  do
7:      $w_j \leftarrow w_j - \text{diff}(x_j, z_j^+)^2 + \text{diff}(x_j, z_j^-)^2$ 
8:   end for
9: end for
10: return 满足  $w_j > t$  的特征集合
```

---

这个算法也是 Filter 的，因为 Filter 的本质是直接选取某种方式，独立的评价某个特征的能力，并打分排序选择。再看 *Relief* 算法，这个算法对每个特征的打分结果就是最终的权重，以某个特征是否可以合理的“推开非同类点”而“拉回同类点”进行打分。

对 *Filter* 算法而言，他并不依赖于具体的模型，不需要在模型上验证，所以是一种相对而言十分高效的算法，一般会先用 *Filter* 进行粗筛，其他算法进行细筛。

**Wrapper 方法 (包裹式)** Wrapper 方法 (包裹式方案) 的核心思想是将后续的学习算法 (如分类器) 直接作为特征评价准则的一部分。在该方案中，学习算法被视为一个“黑盒”，特征选择算法不断地产生不同的特征子集，并将其输入该黑盒中进行训练，最终利用模型在验证集上的性能 (如准确率或错误率) 作为该子集的评价得分。

- **优点:** 针对特定模型“量身定制”，能够捕捉到特征与学习算法之间的相互作用，因此选出的特征子集通常具有更好的预测性能。
- **缺点:** 计算复杂度极高。由于每评估一个子集都需要完整地训练一次模型，在处理高维数据或训练耗时较长的模型时 (如深度学习)，其执行速度非常缓慢。

例 1.5 (LVW (Las Vegas Wrapper) 算法). LVW 是一种基于拉斯维加斯随机化策略 (Las Vegas Algorithm) 的包裹式特征选择方法。其特点是在搜索空间内随机采样特征子集，并通过模型精度来更新当前最优子集。其更

新逻辑不仅关注错误率的降低，还兼顾了特征数量的精简（奥卡姆剃刀原则）。

---

**Algorithm 3** LVW (Las Vegas Wrapper) 算法流程
 

---

**Require:** 数据集  $D$ ，学习算法  $\mathcal{L}$ ，停止准则（控制参数  $T$ ）

**Ensure:** 特征子集  $A^*$

```

1: 初始化：当前最优错误率  $E \leftarrow \infty$ ，最优子集  $A^* \leftarrow A$ （全集），当前
   维度  $d \leftarrow |A|$ ，计数器  $t \leftarrow 0$ 
2: while  $t < T$  do
3:   随机产生特征子集  $A'$ 
4:   计算子集维度  $d' \leftarrow |A'|$ 
5:   通过交叉验证（Cross-Validation）评估学习算法  $\mathcal{L}$  在  $A'$  上的错误
     率  $E'$ 
6:   if  $(E' < E)$  或者  $(E' = E \text{ 且 } d' < d)$  then  $\triangleright$  性能提升或维度更精
     简
7:     更新  $t \leftarrow 0$ 
8:     更新  $E \leftarrow E'$ ， $d \leftarrow d'$ ， $A^* \leftarrow A'$ 
9:   else
10:     $t \leftarrow t + 1$   $\triangleright$  连续  $T$  次尝试无提升则停止
11:   end if
12: end while
13: return 最优特征子集  $A^*$ 

```

---

### 细节说明：

- **停止准则：**算法在连续  $T$  次尝试中未能找到更好的子集时，即认为达到了局部最优或搜索已饱和，从而停止。
- **双重优化目标：**该算法第 9 行的判断逻辑体现了包裹式方案的精髓——首要目标是最小化泛化错误率  $E'$ ，次要目标是在性能持平时尽可能减少特征数量  $d'$ ，以提升模型的可理解性和降低过拟合风险。

## 嵌入式方法

**嵌入式方法概述** 嵌入式方法 (Embedded Methods) 将特征选择过程与学习器的训练过程融为一体，在模型的目标函数中直接引入正则化惩罚项。通过优化带有约束的目标函数，模型在学习过程中会自动调整特征权重，使不重要特征的权重收缩至零，从而实现特征的自动选择。

- **优点：**计算效率高（只需一次训练过程），且由于选择过程直接受模型性能驱动，其准确率通常优于 Filter 方法。
- **缺点：**高度依赖特定的学习器模型（如线性模型），通用性相对较弱。

例 1.6 (LASSO 和岭回归). 一般的损失函数就是误差项，而和是的正则化引入惩罚项目，不仅可以避免过拟合还能实现特征的“稀疏化”去除冗余特征。一般目标函数表示为：

$$\min_w \underbrace{\sum_{i=1}^m (y_i - w^T x_i)^2}_{\text{平方误差项 (RSS)}} + \lambda \underbrace{\|w\|_1}_{\text{L1 惩罚项}}$$

其中  $\lambda$  是调节因子，控制模型在“预测精度”与“特征稀疏性”之间的平衡。

加入惩罚项之后，对误差项影响不大的权重会因为乘法项的存在而减小，事实上，如果存在相关性很高的特征，也可以通过这个算法消除冗余。一个需要关注的问题是，不同的正则项选择会影响算法是否能进行特征的真正系数使得权重  $w = 0$ 。取  $w_1, w_2$  二维权重空间，我们把损失函数分为两个部分，其一是误差项，因为特征相关性图中其等高线是椭圆，其二是正则项，不同正则项的等高线不同。对于某一个特定的误差等高线，它更容易在权重非零的时候与更小的  $L_2$  等高线达到平衡，但对于  $L_1$  等高线，权重更倾向于在坐标轴上达到平衡。

**3. 算法求解：**由于  $L_1$  范数在零点处不可导（尖角特性），无法直接使用标准的梯度下降法。通常采用 **近端梯度下降 (Proximal Gradient Descent, PGD)** 等优化算法，通过软阈值化操作来处理非导数点，确保权重的稀疏性更新。